

OPTIMIZATION ALGORITHMS · MASTER'S COURSE

Foundations of Optimization

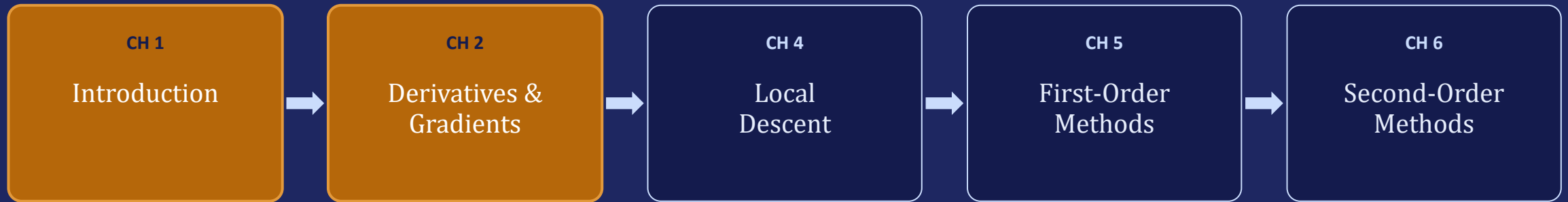
Lecture 1 — What Is Optimization, and How Do We Compute Gradients?

- Optimization basics
- Algorithm landscape
- Gradient extraction

Based on Kochenderfer & Wheeler, Algorithms for Optimization (MIT Press) — Chapters 1 & 2

Course Roadmap

This course follows the structure of Kochenderfer & Wheeler's Algorithms for Optimization. Today covers the foundation everything else builds on.



Today's lecture

Chapters 4, 5 and 6 (local descent, first- and second-order methods) follow directly from the gradient tools we build today, and are the subject of Lecture 2.

PART A

What Is Optimization?

Problem formulation · critical points · the landscape of algorithms

1.1 MOTIVATION

What Is Optimization?

Many disciplines optimize at their core: physics systems settle to lowest-energy states, businesses maximize shareholder value, and organisms evolve for fitness.

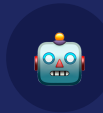
This course takes an engineering perspective: design a system that performs as well as possible on a set of metrics, subject to constraints — cost, weight, safety, structural soundness.

An algorithm is the computational process that searches the space of possible designs for the best one — whether by running physical experiments, evaluating an analytic expression, or simulating.

OPTIMIZATION SHOWS UP IN...



Aerodynamic shape of an aircraft wing



Control policy for an automated vehicle



Detecting cancer from a tumor biopsy image



Sizing the frame of a bicycle

1.1 A HISTORY

Optimization Has Ancient Roots



Euclid (~300 BCE)

Solved early geometric optimization problems — shortest/longest lines, max-area rectangles.



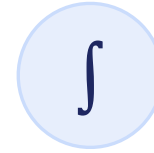
Dido's Problem

Queen Dido enclosed the maximum land area with a fixed-length thong — an isoperimetric optimization.



Descartes (1596–1650)

Introduced coordinates, linking algebra to geometry — the basis for describing design spaces.



Newton & Leibniz

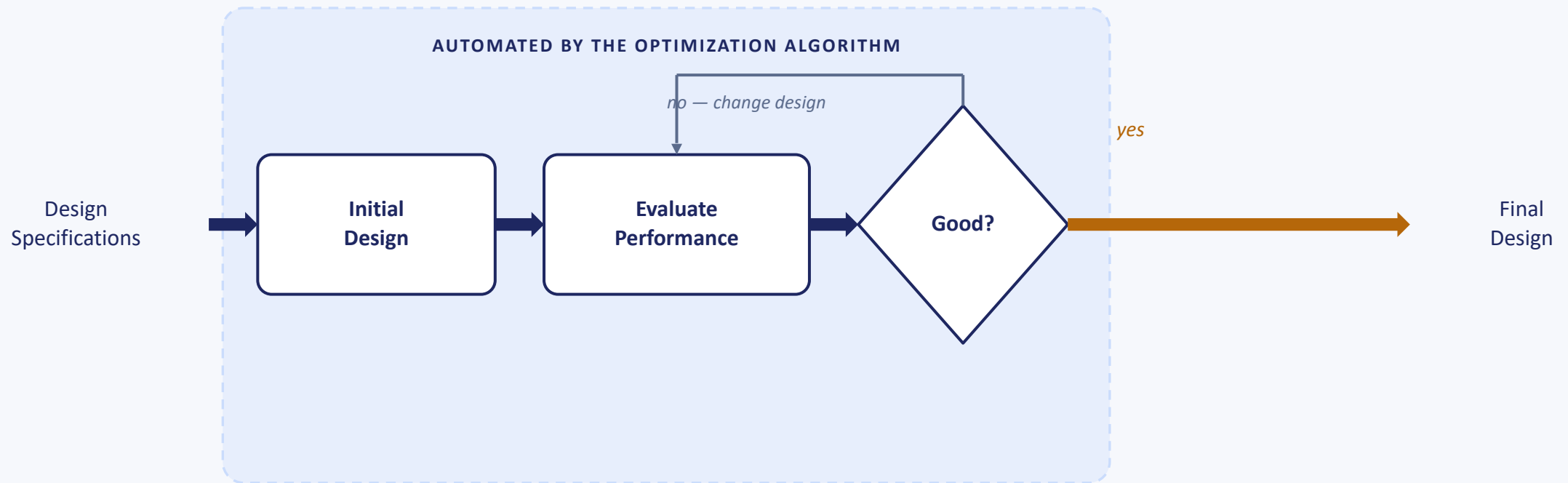
Calculus gave us derivatives — the essential tool for finding where functions are optimal.

The word “algorithm” itself comes from al-Khwārizmī, the 9th-century Persian mathematician who formalized algebra.

1.2 OPTIMIZATION PROCESS

The Design Optimization Loop

The designer supplies a specification — parameters, constants, objectives, constraints — and a baseline design. The algorithm automates the loop highlighted below.



Misspecifications, poor baselines, or an unsuitable algorithm can all lead to suboptimal or unsafe designs — the designer stays responsible for validating results.

1.3 BASIC OPTIMIZATION PROBLEM

Stating the Problem Mathematically

$$\text{minimize}_{\mathbf{x}} \quad f(\mathbf{x}) \quad \text{subject to } \mathbf{x} \in \mathcal{X}$$

$\mathbf{x}$ a design point — a vector of design variables $[x_1, x_2, \dots, x_n]$

f the objective function we want to minimize

$\mathcal{X}$ the feasible set: all points satisfying the constraints

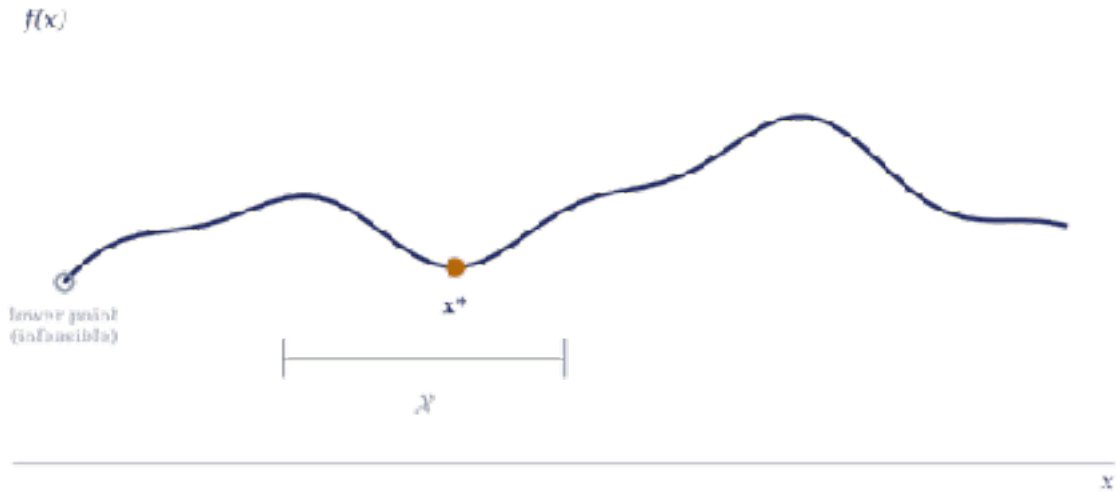
$\mathbf{x}^*$ a minimizer — the solution to the problem

Any maximization problem can be rewritten as a minimization: maximize $f(x)$ is equivalent to minimize $-f(x)$. We can therefore focus on “minimize” without loss of generality.

1.4 CONSTRAINTS

Constraints & the Feasible Set

A one-dimensional optimization problem



Each constraint limits the set of possible solutions. Together they define the feasible set $\mathcal{X}$. Feasible points violate none of the constraints — note the minimum found is only the best within $\mathcal{X}$, not necessarily the global best overall.



THE NO FREE LUNCH THEOREM

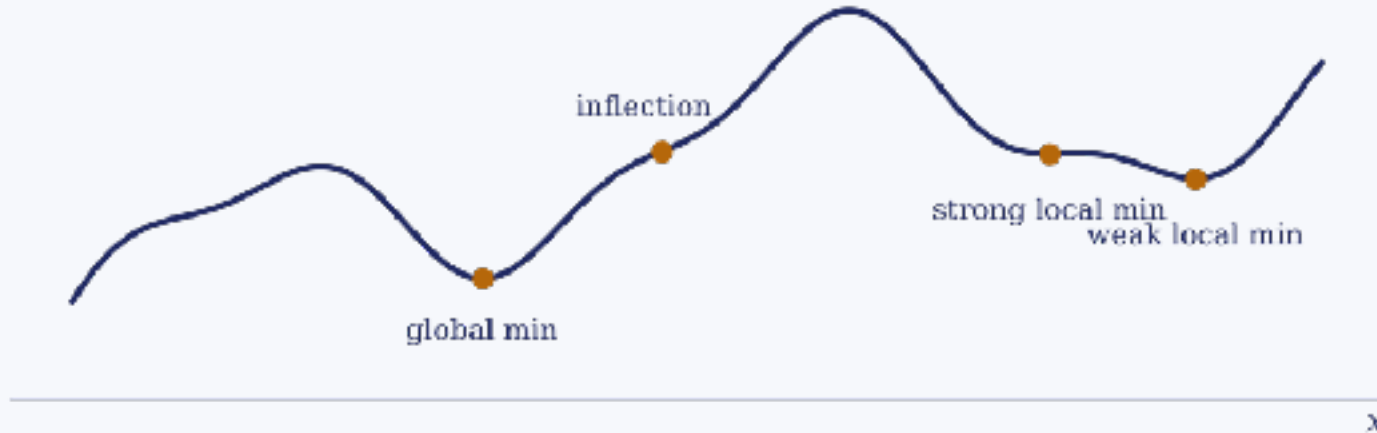
Averaged over all possible objective functions, no algorithm outperforms another. An algorithm that excels on one class of problems necessarily does worse on another.

This is why the rest of the course studies many different algorithms, each suited to different assumptions about the objective function (smoothness, convexity, cost of evaluation...).

1.5 CRITICAL POINTS

Types of Minima

$f(x)$



Global minimum

The point with the lowest objective value across the entire feasible set.

Strong local minimum

Uniquely minimizes f within some neighborhood: $f(x^*) < f(x)$ for all nearby $x \neq x^*$.

Weak local minimum

A local minimizer that is not strong — ties with neighboring points are allowed.

Inflection point

Concavity changes sign here — not an extremum at all, even though the curve briefly flattens.

A zero derivative is necessary for a local minimum of a differentiable function — but not sufficient, as the inflection point shows.

1.6 CONDITIONS FOR LOCAL MINIMA

First- and Second-Order Conditions

Assuming f is differentiable and the point is unconstrained, x^* is guaranteed to be a strong local minimum if:

$$f'(x^*) = 0 \quad (\text{FONC})$$

$$f''(x^*) \geq 0 \quad (\text{SONC})$$

First-Order Necessary Condition (FONC)

$f'(x^*) = 0$ — the point is a stationary point. Shifting x^* by a small amount does not change f to first order.

Second-Order Necessary Condition (SONC)

$f''(x^*) \geq 0$ — the function curves upward (or is flat) around x^* , consistent with the bottom of a bowl.

These conditions are necessary, not sufficient: they must hold at any local minimum, but not every point satisfying them is one (recall the inflection point).

Gradients & Hessians

In multiple dimensions, the first derivative generalizes to the gradient — a vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

The second derivative generalizes to the Hessian — a matrix of second partial derivatives:

$$\nabla^2 f(\mathbf{x})_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

At a local minimum, the gradient vanishes ($\nabla f = 0$) and the Hessian is positive semidefinite — the multivariate analogue of FONC and SONC. Computing these efficiently is exactly the subject of Part B.

WORKED EXAMPLE

Checking FONC & SONC: the Rosenbrock Function

$$f(\mathbf{x}) = (1 - x_1)^2 + 5(x_2 - x_1^2)^2$$

A classic, notoriously curved test function for optimization algorithms — minimized at $\mathbf{x}^* = (1, 1)$.

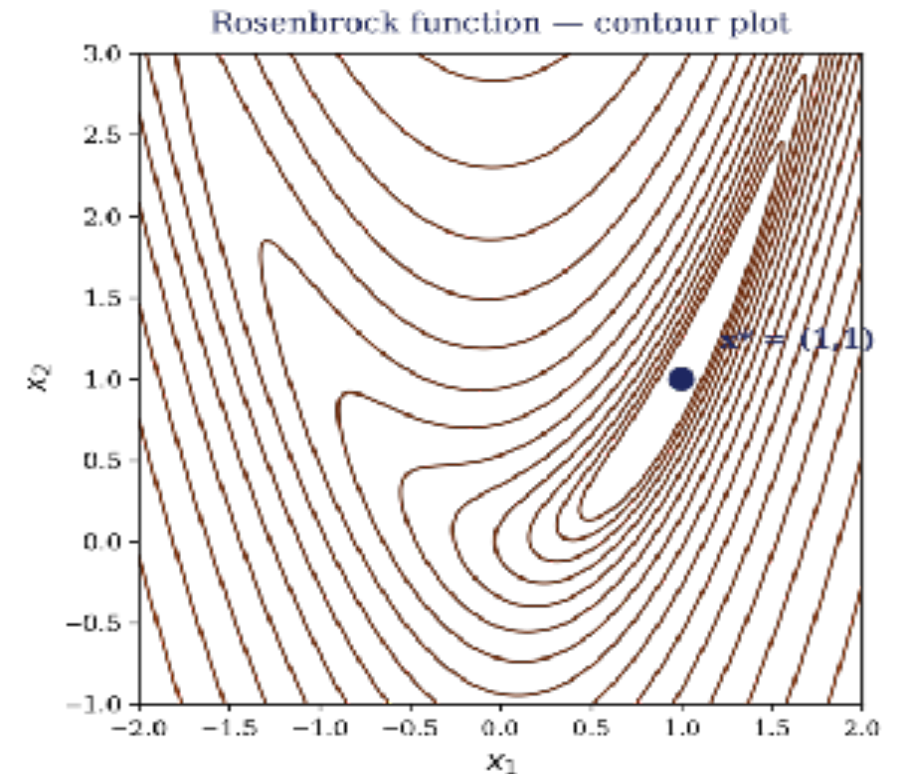
$$\nabla f(\mathbf{x}) = (2(10x_1^3 - 10x_1x_2 + x_1 - 1), 10(x_2 - x_1^2))$$

● $\nabla f(1, 1) = (0, 0)$

The gradient vanishes at $(1, 1)$ — FONC is satisfied.

● $\nabla^2 f(1, 1) = \begin{bmatrix} 42 & -20 \\ -20 & 10 \end{bmatrix}$

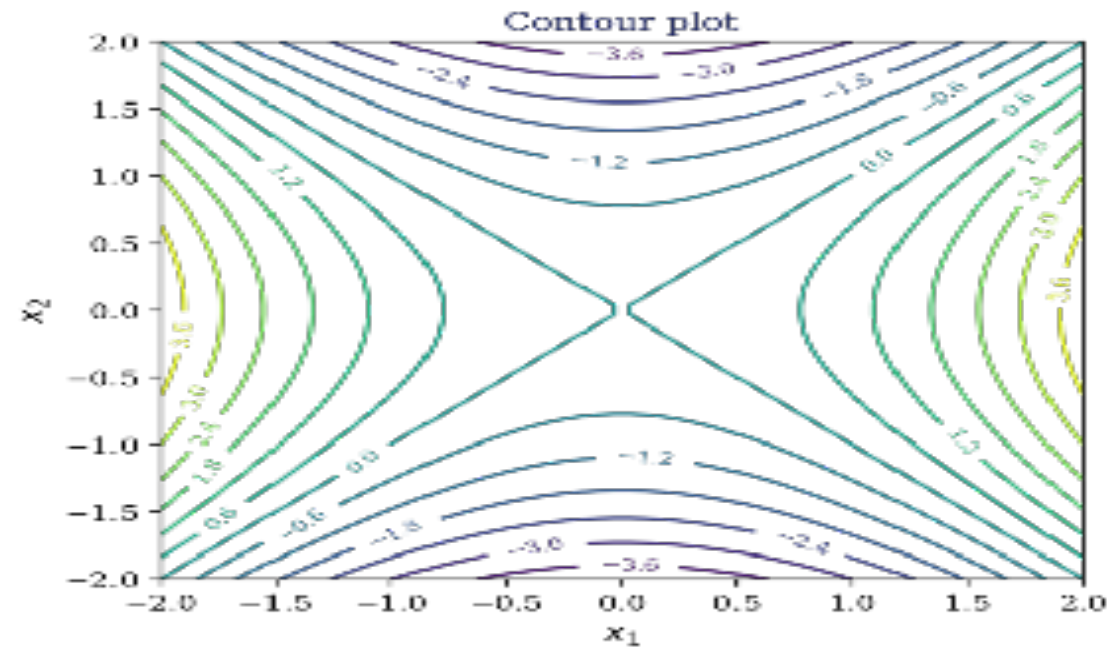
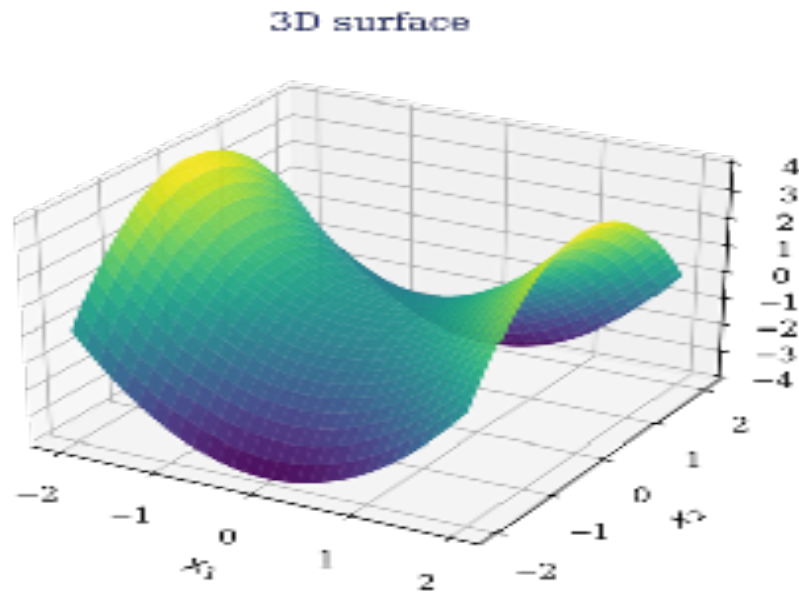
This Hessian is positive definite — SONC is satisfied. $(1, 1)$ is confirmed a strong local (in fact global) minimum.



1.7 CONTOUR PLOTS

Visualizing Multivariate Functions

A function of two variables, $f(x_1, x_2)$, can be visualized as a 3D surface — or, more practically, as a contour plot: lines of constant f value, like elevation lines on a map.



1.8 OVERVIEW

The Landscape of Optimization Algorithms

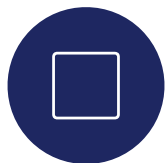
Since no single algorithm dominates (no free lunch), the book organizes methods into four families, each making different assumptions about what information is available.



Gradient-Based

Ch. 4–6

Use derivative information to choose a descent direction (first-order) and optionally curvature (second-order).



Direct / Gradient-Free

Ch. 7

Search using only function evaluations — pattern search, simplex methods, Lipschitz-based approaches.



Stochastic

Ch. 8

Inject randomness into the search — e.g. noisy descent, simulated annealing, cross-entropy methods.



Population

Ch. 9

Evolve a collection of candidate points together — genetic algorithms, particle swarm optimization.

Today (Part B) we build the gradient toolbox — exact and approximate derivatives — that first- and second-order methods rely on.

Gradient-Based Methods



Use $\nabla f(x)$ — and sometimes $\nabla^2 f(x)$ — to decide which direction to move at every iteration.

First-order methods (Ch. 5)

Step in a direction informed by the gradient — e.g. steepest descent, momentum, conjugate gradient. Cheap per step, but can zig-zag on ill-conditioned problems.

Second-order methods (Ch. 6)

Also use curvature (the Hessian) to inform both direction and step size — e.g. Newton's method. Fewer iterations, but each step is more expensive.

The catch: gradient-based methods need a gradient. When f is a black box, a simulation, or otherwise not analytically differentiable, we must estimate ∇f — the problem Part B solves.

Population & Stochastic Methods



Population Methods (Ch. 9)

- Maintain a whole collection of candidate points, not just one.
- Spreading points across the design space reduces the risk of getting stuck in a local minimum.
- Relies on stochasticity to keep the population diverse; examples include genetic algorithms and particle swarm optimization.
- Can be combined with local (gradient-based) descent for refinement.



Stochastic Methods (Ch. 8)

- Incorporate randomness directly into the search process.
- Can improve on deterministic algorithms from earlier chapters — e.g. noisy descent, randomized pattern search.
- Some methods learn a probability distribution over the design space, weighting regions more likely to contain the optimum.
- Useful when the objective is noisy or has many local minima.

Part A — Key Takeaways

- 1 Optimization = search for x^* minimizing $f(x)$ subject to $x \in X$.
- 2 FONC ($\nabla f = 0$) and SONC (Hessian positive semidefinite) are necessary conditions for a local minimum.
- 3 No Free Lunch: no algorithm dominates across all problems — hence many algorithm families.
- 4 Four families: gradient-based, direct/gradient-free, stochastic, population.
- 5 Gradient-based methods (our focus going forward) need a gradient — up next: how to get one.

PART B

Derivatives & Gradient Extraction

Finite differences · the complex-step trick · automatic differentiation

2.1 MOTIVATION

Why Do We Need Derivatives?

Gradient-based algorithms use $\nabla f(x)$ to choose a descent direction. But in practice, we often can't write down ∇f analytically:



f is the output of a simulation or physical experiment, not a formula.



f 's source code is enormous, legacy, or opaque — hand-deriving ∇f is impractical.



f may be a black box — only function evaluations are available.

This chapter covers four ways to get a derivative: exact symbolic reasoning aside, we'll estimate it numerically (finite differences, complex step) or compute it exactly via automatic differentiation.

2.3 NUMERICAL DIFFERENTIATION

Finite Differences: the Basic Idea

All finite-difference methods approximate $f'(x)$ using nearby function evaluations, derived from the Taylor expansion:

$$f(x+h) = f(x) + \frac{f'(x)}{1!}h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \dots$$

Rearranging this expansion in different ways yields three classic estimators, trading off cost and accuracy:

Forward difference

uses: $f(x+h)$, $f(x)$

error $O(h)$

Central difference

uses: $f(x+h/2)$, $f(x-h/2)$

error $O(h^2)$

Backward difference

uses: $f(x)$, $f(x-h)$

error $O(h)$

2.3.1 FINITE DIFFERENCES

Forward Difference

Truncate the Taylor expansion of $f(x+h)$ after the linear term and solve for $f'(x)$:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \quad \mathcal{O}(h)$$

```
diff_forward(f, x; h) = (f(x+h) - f(x)) / h
```

- Requires only one extra function evaluation, $f(x+h)$ — the cheapest option.
- The truncated terms contribute error proportional to h , so the error shrinks linearly as $h \rightarrow 0$ — hence $\mathcal{O}(h)$.
- In practice h is chosen around the square root of machine precision, balancing truncation error against floating-point round-off.

2.3.1 FINITE DIFFERENCES

Central Difference — a Better Trade-off

Evaluate symmetrically around x . The linear error terms cancel when the two Taylor expansions are subtracted, leaving quadratic error:

$$f'(x) \approx \frac{f(x+h/2) - f(x-h/2)}{h} \quad \mathcal{O}(h^2)$$

```
diff_central(f, x; h) = (f(x+h/2) - f(x-h/2)) / h
```

Twice the cost of forward difference (two evaluations away from x instead of one reused $f(x)$), but the error shrinks quadratically — $\mathcal{O}(h^2)$ instead of $\mathcal{O}(h)$. Halving h cuts the error roughly 4× instead of 2×.

2.3.1 FINITE DIFFERENCES

Backward Difference & the Step-Size Dilemma

Backward difference

$$f'(x) \approx \frac{f(x) - f(x-h)}{h} \quad \mathcal{O}(h)$$

Mirror image of the forward difference, stepping backward instead — same $\mathcal{O}(h)$ accuracy and cost. Useful when $f(x+h)$ is unavailable (e.g. at a domain boundary).

The dilemma: how small should h be?

Too large

Truncation error dominates — the Taylor approximation itself is poor (the $\mathcal{O}(h)$ / $\mathcal{O}(h^2)$ terms we dropped are large).

Too small

Subtractive cancellation: $f(x+h)$ and $f(x)$ become nearly identical in floating point, and their difference loses precision to round-off.

There is a sweet spot in between — but it depends on f , and getting it wrong costs accuracy either way. The complex-step method sidesteps this entirely.

2.3.2 COMPLEX STEP METHOD

Sidestepping Cancellation with a Complex Step

Idea: step in the imaginary direction instead of the real one. The Taylor expansion of $f(x+ih)$ separates cleanly into real and imaginary parts:

$$f(x + ih) = f(x) + ihf'(x) - h^2 \frac{f''(x)}{2!} - ih^3 \frac{f'''(x)}{3!} + \dots$$

Taking just the imaginary component isolates the derivative — with no subtraction of nearly-equal numbers:

$$f'(x) = \frac{\text{Im}(f(x + ih))}{h} + \mathcal{O}(h^2)$$

Only one function evaluation is needed (at a complex argument), the error is $\mathcal{O}(h^2)$ like central difference, and there is no cancellation — so h can be shrunk far below what forward/central differences tolerate, all the way to machine precision.

2.4 AUTOMATIC DIFFERENTIATION

Differentiating Programs Exactly

A program is built from elementary operations (+, −, ×, ÷, log, sin, ...). Automatic differentiation (AD) applies the chain rule mechanically to every operation:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \frac{dg}{dx}$$

Unlike finite differences, AD is not an approximation — it computes the derivative to machine precision, in principle exactly, by composing exact local derivative rules.

Forward accumulation

Traverses the computational graph from inputs to outputs in a single pass, propagating both value and derivative together (“dual numbers”).

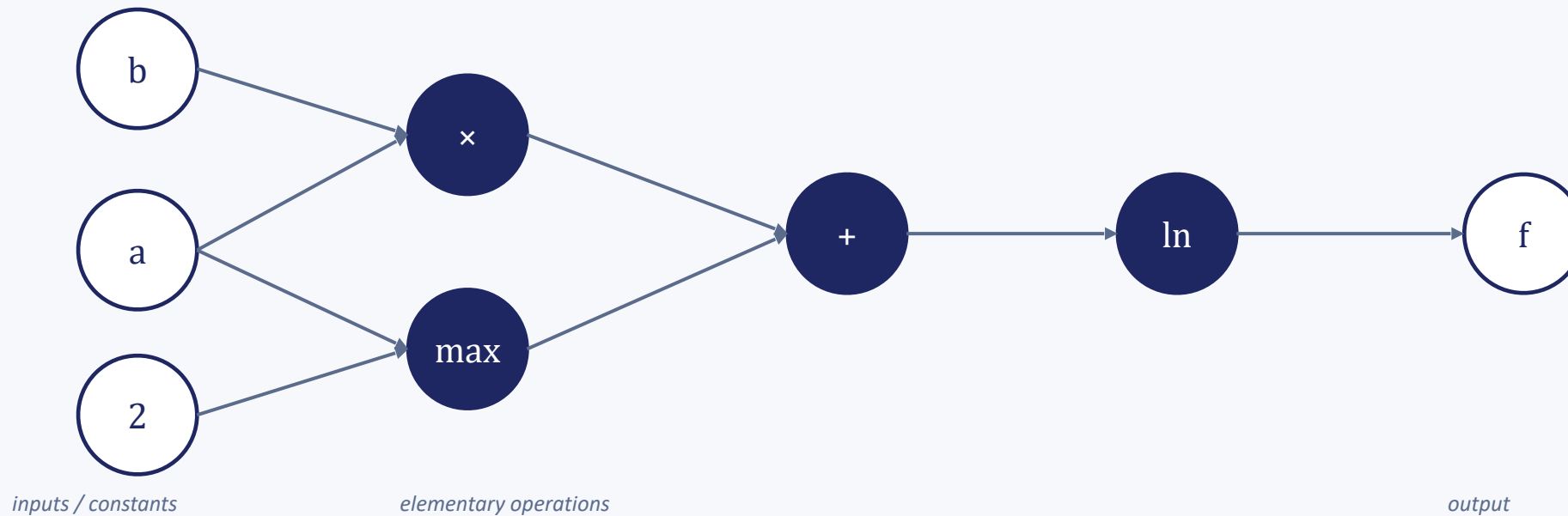
Reverse accumulation

Runs forward to compute values, then sweeps backward through the graph to accumulate derivatives — the basis of backpropagation in deep learning.

2.4 AUTOMATIC DIFFERENTIATION

Example: A Computational Graph

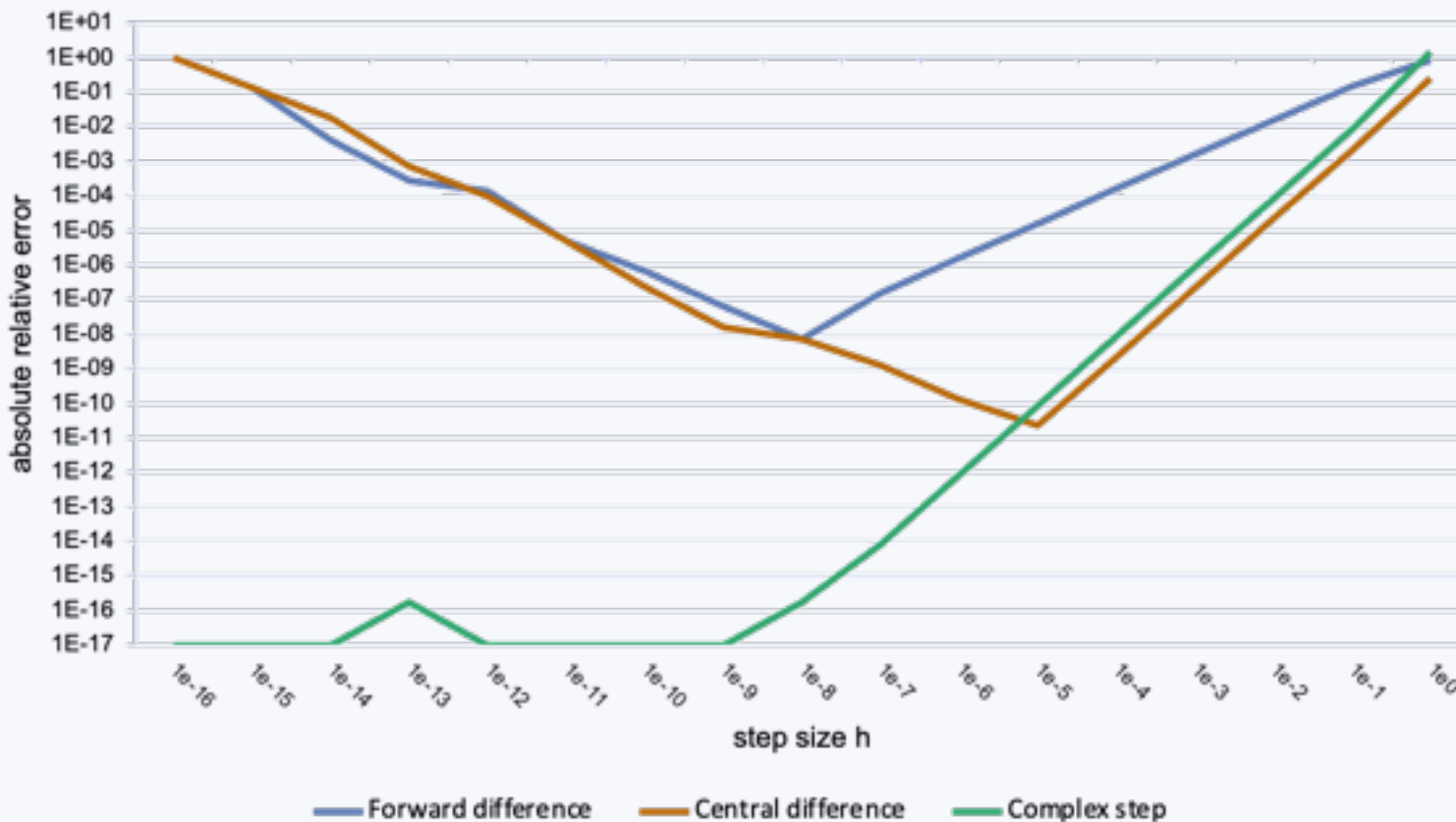
$f(a, b) = \ln(a \cdot b + \max(a, 2))$ — broken into elementary operations:



Leaf nodes are inputs/constants; each internal node applies one elementary operation. Forward accumulation walks left-to-right, propagating value AND partial derivative through every node in one pass.

Comparing Derivative Estimates

Estimating $f'(\pi/2)$ for $f(x) = \sin(x^2)$, true value ≈ -2.45425 , across step sizes h from 10^{-16} to 1:



Forward

Linear $O(h)$ slope on the right; noisy floor from round-off on the left.

Central

Quadratic $O(h^2)$ slope — steeper improvement as h shrinks, before round-off dominates.

Complex step

Flat at machine precision ($\sim 10^{-16}$) across many orders of magnitude — no cancellation floor at all.

Which Method, When?

Method	Evaluations	Error order	Cancellation risk	Best when...
Forward difference	2 (reuses $f(x)$)	$O(h)$	Moderate	Quick estimate; f is cheap and smooth
Central difference	2	$O(h^2)$	Moderate	Better accuracy needed, still black-box f
Complex step	1 (complex arg)	$O(h^2)$	None	f supports complex arithmetic; near machine precision needed
Automatic differentiation	1 pass	Exact	None	Source code available; needed for large-scale gradients

Rule of thumb: prefer AD when you can (exact, one pass); fall back to complex step when you need a numeric estimate without cancellation; use central/forward differences for quick, dependency-free checks.

Part B — Key Takeaways

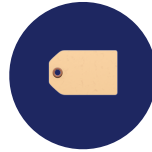
- 1 Gradient-based methods need ∇f — often unavailable analytically, so we estimate or compute it.
- 2 Finite differences (forward, central, backward) trade cost for accuracy: $O(h)$ vs $O(h^2)$.
- 3 Step size is a dilemma: too large $\rightarrow$ truncation error; too small $\rightarrow$ subtractive cancellation.
- 4 The complex-step method removes cancellation entirely, reaching near machine precision.
- 5 Automatic differentiation computes exact derivatives by mechanically applying the chain rule through a computational graph (forward or reverse accumulation).

Putting the Pieces Together



The problem

minimize $f(x)$ subject to $x \in X$, with FONC/
SONC characterizing local minima.



The landscape

Gradient-based, direct, stochastic, and
population methods — no single winner (No
Free Lunch).



The gradient toolbox

Finite differences, complex step, and
automatic differentiation — four ways to get
 ∇f .

Together, these give us everything a first-order descent method needs: a well-posed problem, and a reliable gradient.

Next Time: Local Descent

With a way to compute gradients in hand, Lecture 2 turns to using them to actually find a minimum:

Ch. 4

Local Descent

The general iterate-and-step framework: choose a direction, choose a step length, repeat.

Ch. 5

First-Order Methods

Steepest descent, momentum, conjugate gradient — using ∇f to guide each step.

Ch. 6

Second-Order Methods

Newton's method and quasi-Newton methods — using curvature for faster convergence.

Questions?

Reading for next time: Algorithms for Optimization, Chapters 4 & 5